ASTERIA

SYSTEM ARCHITECTURE

Host-Side Robot Control Stack for VEX AIM

DAEMON | GUI | 3DS CLIENT | OPENCLAW | VEX AIM

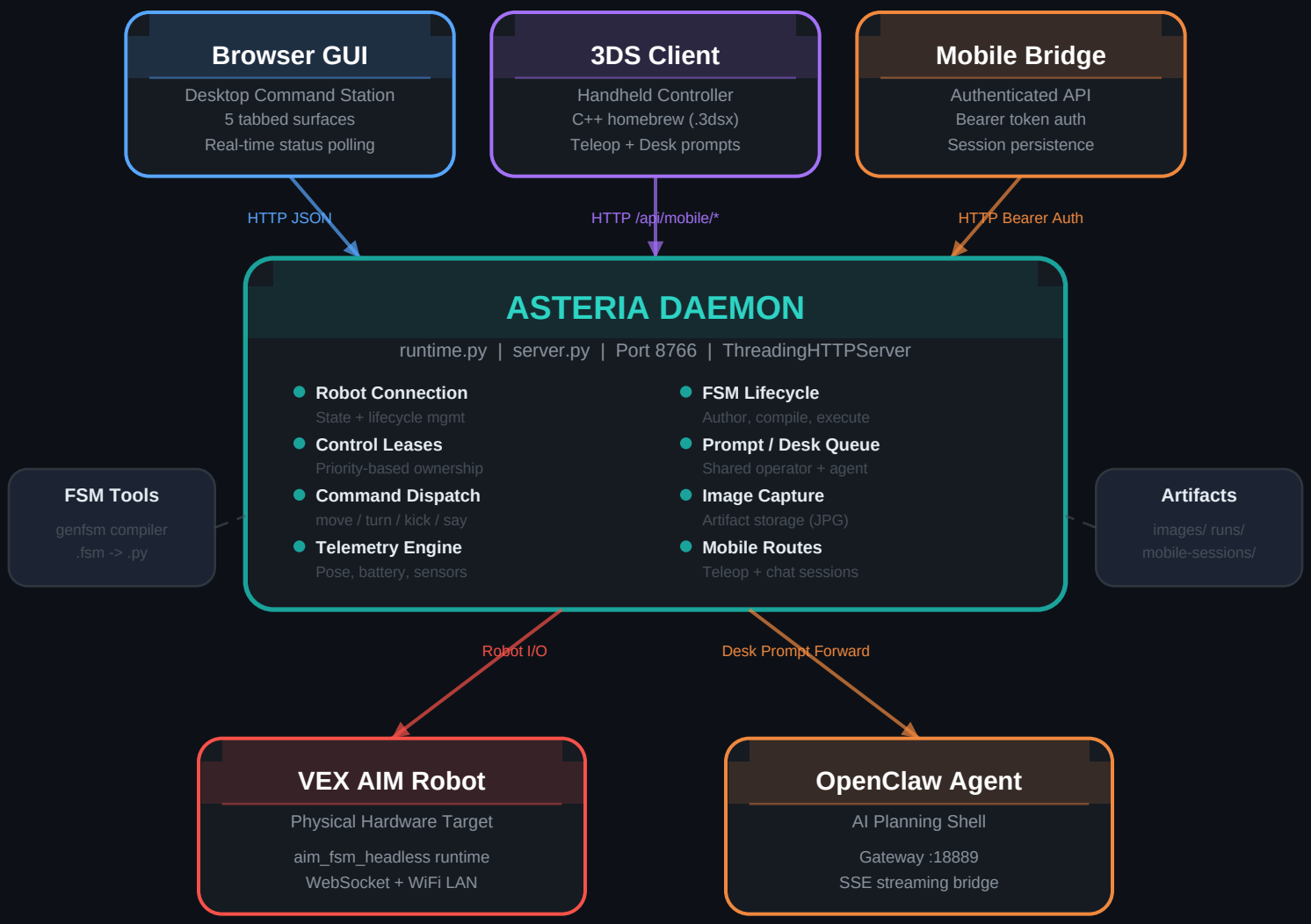
Carnegie Mellon University | Cognitive Robotics | Spring 2026

Noah Hicks | MS AI Systems Management

v1.0 APR 2026

System Architecture

How Asteria's components connect and communicate



LEGEND: Operator Surface · Agent Integration · Hardware Target · Support Module

The Daemon

Central runtime and single source of truth for all robot operations

CORE SPECIFICATIONS

Server	**ThreadingHTTPServer**
Port	**8766 (localhost)**
Entry Point	**python -m asteria.daemon.server**
Runtime	**AsteriaRuntime state machine**
Core Files	**runtime.py (2229 lines)**
Language	**Python 3.x**
Platform	**Windows-first**
Data Classes	**ControlLease, TelemetrySnapshot**

RUNTIME RESPONSIBILITIES

- Robot connection state & lifecycle management
- Control lease ownership with priority handoff
- Direct command execution (move/turn/kick/say)
- FSM file creation, compilation, and execution
- Image capture and artifact storage
- Telemetry and pose tracking
- Prompt/Desk queue with OpenClaw bridge
- Mobile bridge authentication & routing

HTTP API Endpoints

METHOD	ENDPOINT	DESCRIPTION
GET	**/api/status**	Full daemon + robot status snapshot
POST	**/api/command**	Execute direct robot command
POST	**/api/lease/claim**	Claim control lease (priority-aware)
POST	**/api/lease/release**	Release control lease
GET	**/api/images**	List captured image artifacts
GET	**/api/fsms**	List FSM files and compilation state
GET	**/health**	Daemon health check
POST	**/api/mobile/teleop/vector**	Send teleop movement vector
POST	**/api/mobile/chat/sessions**	Create mobile chat session
GET	**/api/mobile/bootstrap**	Mobile session bootstrap

CONTROL LEASE POLICY

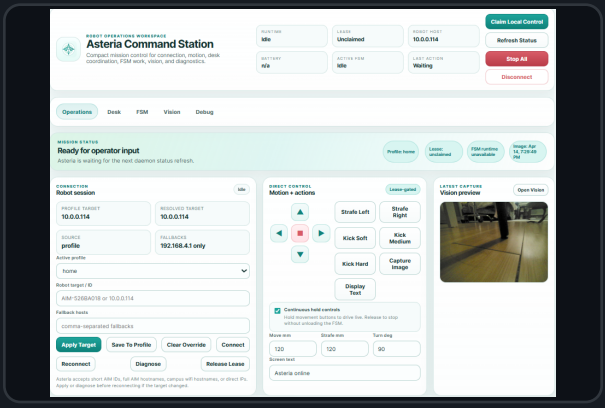
Single active holder (human or agent). Humans cannot be preempted by agents.
Agent holders can be overridden with --force. Leases auto-expire; activity refreshes timeout.

Operator Surfaces

Three distinct interfaces for human control and monitoring

Browser GUI

Desktop Command Station | index.html + app.js + styles.css



Operations

Connection targets, motion controls, telemetry, battery, active FSM status

Desk

Shared operator/agent prompts, activity feed, notes, response resolution

FSM

File editor, compile/run controls, event injection, compiled state

Vision

Large image viewer, telemetry snapshot, command history

Debug

Connection diagnostics, raw JSON status, stop policy, command log

Old-3DS Client

Handheld homebrew controller

Binary	**asteria-ds.3dsx (7.3 MB)**
Language	**C++17, libctru, citro2d**
Build	**devkitPro / Makefile**
Config	**sdmc:/3ds/asteria-ds/config.json**
Audio	**NDSP: theme, boot, press, tab**
Protocol	**HTTP to /api/mobile/***

Mobile Bridge

Authenticated API surface within daemon

Auth	**Bearer token (constant-time)**
Sessions	**Persistent JSON storage**
Teleop	**Joystick vector -> discrete pulses**
Deadzone	**0.18 magnitude threshold**
Rate Limit	**80ms between commands**
Preview	**RGB565 image thumbnails**

KEY DESIGN: All surfaces are pure clients. The daemon is the single source of truth. No parallel state management.

Integrations

OpenClaw agent bridge and VEX AIM robot target

OpenClaw Agent Bridge

AI planning shell connected through Windows via local gateway

Bridge File	**openclaw_bridge.py (~700 lines)**
Gateway URL	**http://127.0.0.1:18889**
API Path	**/v1/responses (SSE stream)**
Model	**openclaw/default**
Session Key	**session:asteria-desk:{prompt_id}**
Timeout	**120,000ms**
User Identity	**asteria-desk-bridge**
Auto-Start	**Launches gateway if health check fails**

BRIDGE FLOW

1. Desk prompt created in Asteria
2. Background thread forwards to Gateway
3. SSE stream parsed for events
4. On acceptance: forward_status = sent
5. Resolution separate from forwarding
6. Failed forwards backed off & retried

WIRE FORMAT Session routing via X-OpenClaw-Session-Key header (NOT JSON body)

Events: response.created | response.in_progress | response.completed | response.failed

VEX AIM Robot

Physical hardware target with dual runtime modes

Primary Runtime	**aim_fsm_headless (FSM + events)**
Fallback Runtime	**aim_raw_minimal (commands only)**
Connection	**WiFi LAN / WebSocket**
Tools Repo	**vex-aim-tools/ (local clone)**
FSM Compiler	**genfsm (.fsm -> .py)**

DIRECT COMMANDS

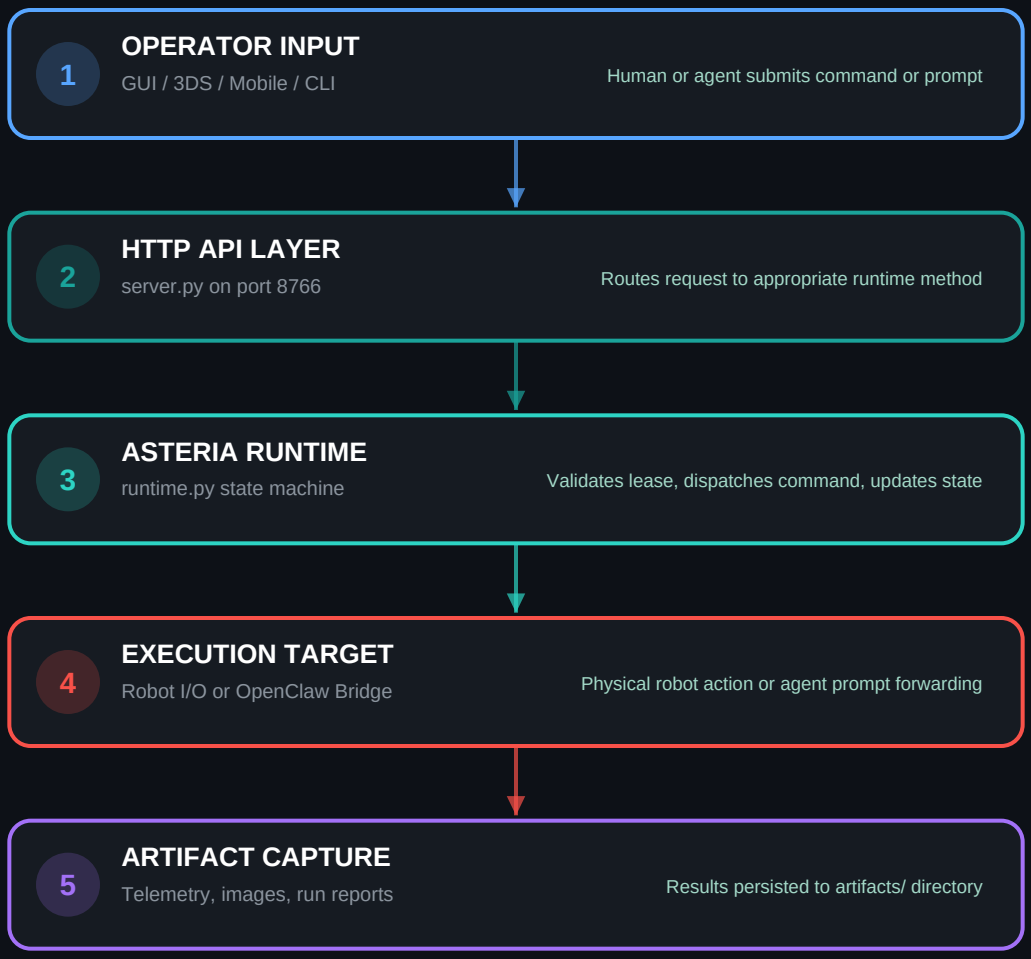
- move
- turn
- sideways
- kick
- say
- stop_all

ARTIFACT STORAGE images/*.jpg | fsm-sources/*.fsm + *.py | runs/*.json | mobile-sessions/ | mobile-previews/

All artifacts written by daemon under artifacts/ directory. FSM source is source of truth; compiled .py is generated.

Data Flow & CLI

How data moves through the system and programmatic access



CLI Interface

cli.py | Default: --base-url=http://127.0.0.1:8766

status	Read daemon status	**claim-lease**	Claim control (--force)
connect	Connect to robot	**stop-all**	Emergency stop
capture-image	Capture image	**submit-prompt**	Post desk prompt
create-fsm	Create FSM file	**compile-fsm**	Compile FSM
run-fsm	Execute FSM	**diagnose**	Inspect connection

STARTUP: start_asteria.ps1 -OpenBrowser

or: python -m asteria.daemon.server